

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
zip_path = "/content/drive/MyDrive/Rice_Image_Dataset.zip"
```

```
import zipfile

dataset_path = "/content/Rice_Image_Dataset/Rice_Image_Dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(dataset_path)

print("Extraction Done!")
```

Extraction Done!

```
import os

print(os.listdir(dataset_path))

['Karacadag', 'Jasmine', 'Rice_Image_Dataset', 'Basmati', 'Ipsala', 'Rice_Citation_Request.txt', 'Arborio']
```

```
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# -----
# STEP 1: PATHS
# -----
class1_path = "/content/Rice_Image_Dataset/Rice_Image_Dataset/Basmati"
class2_path = "/content/Rice_Image_Dataset/Rice_Image_Dataset/Jasmine"

images = []
labels = []
```

```
# -----
# STEP 2: LOAD 200 IMAGES EACH
# -----
def load_images(folder, label):
    files = os.listdir(folder)
    count = 0

    for file in files:
        if count == 200:
            break

        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path)

        if img is not None:
            img = cv2.resize(img, (128, 128))
            images.append(img)
            labels.append(label)
            count += 1

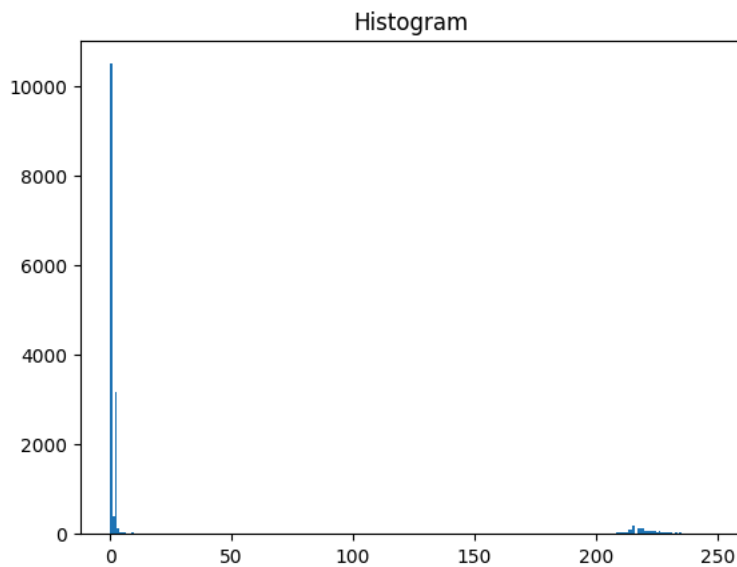
# Load images
load_images(class1_path, 0)
load_images(class2_path, 1)

print("Total images loaded:", len(images))
```

Total images loaded: 400

```
# -----
# STEP 3: HISTOGRAM
# -----
gray = cv2.cvtColor(images[0], cv2.COLOR_BGR2GRAY)

plt.hist(gray.ravel(), bins=256)
plt.title("Histogram")
plt.show()
```



```
# -----
# STEP 4: FEATURE EXTRACTION
# -----
features = []

for img in images:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if len(contours) > 0:
        cnt = max(contours, key=cv2.contourArea)

        area = cv2.contourArea(cnt)
        perimeter = cv2.arcLength(cnt, True)

        compactness = (perimeter ** 2) / area if area != 0 else 0

        if len(cnt) >= 5:
            ellipse = cv2.fitEllipse(cnt)
            (center, axes, orientation) = ellipse
            major_axis = max(axes)
            minor_axis = min(axes)
            eccentricity = np.sqrt(1 - (minor_axis / major_axis) ** 2)
        else:
            eccentricity = 0

        features.append([area, perimeter, compactness, eccentricity])
```

```
# -----
# STEP 5: CREATE DATASET
# -----
df = pd.DataFrame(features, columns=[
    "Area", "Perimeter", "Compactness", "Eccentricity"
])

df["Label"] = labels

print("\nTotal rows:", len(df))
print("\nClass count:\n", df["Label"].value_counts())
```

```
Total rows: 400

Class count:
Label
0      200
1      200
Name: count, dtype: int64
```

```
# -----
# STEP 6: DISPLAY DATASET
# -----
```

```

print("\n First 10 rows of dataset:\n")
print(df.head(10))

print("\n Last 10 rows of dataset:\n")
print(df.tail(10))

print("\n Full Dataset:\n")
print(df)

print("\n Dataset Shape (rows, columns):")
print(df.shape)

print("\n Statistical Summary:\n")
print(df.describe())

print("\n Class Distribution:\n")
print(df["Label"].value_counts())

```

```

5  1373.0  185.740114  25.127014  0.971201  0
6  1981.5  243.948266  30.033185  0.980148  0
7  1710.0  209.882250  25.760561  0.970235  0
8  2250.5  253.320848  28.514309  0.976701  0
9  1638.5  212.007140  27.431814  0.969929  0

```

Last 10 rows of dataset:

	Area	Perimeter	Compactness	Eccentricity	Label
390	1328.5	165.722870	20.672992	0.937007	1
391	1388.5	183.154328	24.159530	0.955063	1
392	2342.0	231.539104	22.890844	0.952785	1
393	2092.0	205.882250	20.261712	0.930347	1
394	1300.0	173.965509	23.279999	0.955392	1
395	2465.5	232.953318	22.010646	0.948880	1
396	2282.0	221.078210	21.417868	0.950133	1
397	1229.0	168.852812	23.198757	0.955245	1
398	2247.0	230.107646	23.564543	0.959878	1
399	1245.5	165.840619	22.081984	0.949399	1

Full Dataset:

	Area	Perimeter	Compactness	Eccentricity	Label
0	1783.5	200.350286	22.506441	0.948817	0
1	1744.5	224.835567	28.977376	0.978070	0
2	1870.0	222.735062	26.529897	0.973459	0
3	1986.0	242.710677	29.661869	0.977403	0
4	1768.5	220.634557	27.525930	0.976665	0
..	...	...	...	...	...
395	2465.5	232.953318	22.010646	0.948880	1
396	2282.0	221.078210	21.417868	0.950133	1
397	1229.0	168.852812	23.198757	0.955245	1
398	2247.0	230.107646	23.564543	0.959878	1
399	1245.5	165.840619	22.081984	0.949399	1

[400 rows x 5 columns]

Dataset Shape (rows, columns):
(400, 5)

Statistical Summary:

	Area	Perimeter	Compactness	Eccentricity	Label
count	400.000000	400.000000	400.000000	400.000000	400.000000
mean	1658.008750	203.753473	25.308469	0.962648	0.500000
std	375.269341	30.014685	3.331857	0.014294	0.500626
min	1202.500000	155.396968	19.272114	0.920065	0.000000
25%	1321.000000	172.771337	22.431136	0.952100	0.000000
50%	1607.750000	210.187336	24.892775	0.963872	0.500000
75%	1927.750000	230.618053	27.884270	0.975150	1.000000
max	2791.500000	261.806130	36.140669	0.986970	1.000000

Class Distribution:

```

Label
0    200
1    200
Name: count, dtype: int64

```

```

# -----
# STEP 7: SAVE TO CSV
# -----
csv_path = "/content/rice_dataset.csv"

df.to_csv(csv_path, index=False)

print("\nDataset saved to CSV!")

```

Dataset saved to CSV!

```
# -----
# STEP 8: LOAD CSV & SPLIT
# -----
from sklearn.model_selection import train_test_split

data = pd.read_csv(csv_path)

X = data[["Area", "Perimeter", "Compactness", "Eccentricity"]]
y = data["Label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Training samples: 280
Testing samples: 120

```
# -----
# STEP 9: FEATURE SCALING
# -----
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# -----
# STEP 10: TRAIN KNN
# -----
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=11)
knn.fit(X_train, y_train)

print("\nModel trained successfully!")
```

Model trained successfully!

```
# -----
# STEP 11: PREDICTION & METRICS
# -----
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = knn.predict(X_test)

acc = accuracy_score(y_test, y_pred) * 100
print("\nAccuracy: {:.2f}%".format(acc))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 96.67%

Confusion Matrix:
[[60 2]
[2 56]]

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.97	0.97	62
1	0.97	0.97	0.97	58
accuracy			0.97	120
macro avg	0.97	0.97	0.97	120
weighted avg	0.97	0.97	0.97	120

